

Training Improvements

Augmentation changes and their measured effect

These changes target two known weak points (multi-defect fusion and noise robustness) by enriching the **training data**, not the model. The SCN-Attention U-Net architecture and the fixed scattering filters are **unchanged** — so there is no risk of the collapse a redesign would cause. The production checkpoint was trained with these augmentations (composite + noise) applied to the **train split only**; validation and test data stay clean.

What was added (`configs/model.yaml` → `augment`, **train split only**)

Knob	Value	Fixes	How it helps
<code>noise_p / noise_sigma</code>	0.3 / 0.04	Noise robustness	Adds additive Gaussian noise to training images so the seg model learns to keep finding defects under acquisition noise.
<code>composite_p</code>	0.25	Multi-defect fusion	With 25% probability, a training image is max-blended with one of the other class and their masks are unioned — so the model sees both porosity and slag in one image and learns to segment both. Every original image is single-type, which is exactly why fusion detection was weak.

Both are train-only (never val/test) and gated by config (default off → no behaviour change unless enabled). Verified: with `composite_p=1.0` every sampled training mask contains both flaw classes.

What this does NOT fix

- **Porosity Dice (~0.66):** porosity remains the weaker class relative to slag (~0.71). The root

cause is the pseudo-labels, which shrink porosity to tiny cores; the real lever is more hand-labelling. Composite/noise augmentation helps a little but is not the main fix.

- **Extreme-noise type accuracy:** a fundamental signal-to-noise limit (see the robustness probe

results / `scripts/robustness.py`) — not addressed by training augmentation.

How to reproduce the training run

```
# in the Colab notebook (notebooks/train_colab.ipynb) – the augmentation
# is read from configs/model.yaml automatically:
py -3.14 -m src.models.train --config configs/model.yaml
# then re-evaluate and refresh the experiment numbers:
py -3.14 -m src.models.evaluate --ckpt checkpoints/scn_attn_unet_best.pt --split test
py -3.14 -m scripts.multidefect_fusion --pairs 8      # both-detected reflects the fusion training
```

```
py -3.14 -m scripts.robustness --limit 10          # the noise curve falls less steeply
```

These commands reproduce the deployed checkpoint: training writes the checkpoint, and re-evaluation regenerates `model_metrics.json` and the experiment outputs that ship with it.